UNIVERSIDAD: Politécnica de Atlautla

PROGRAMA: Ingeniería en Tecnologías de la Información e Innovación Digital

ASIGNATURA: Fundamentos de Inteligencia Artificial

DOCENTE: Mtro. Javier García Lara

Modelado formal de dos problemas de búsqueda y construcción de sus espacios de búsqueda con NetworkX

SECCIONES: 2.1 Problemas de búsqueda / 2.2 Espacios de búsqueda

ALUMNO: JAXIE EDREY SANCHEZ HERNANDEZ | FECHA: 18 de octubre de 2026

ARCHIVO: FIA_172_Sancchez _S2.pdf

Herramienta: Venice AI. **Uso:** andamiaje y revisión de la formulación. **Verificación:** se ejecutó el código y se contrastaron los conteos de estados contra los valores teóricos ($9!/2 = 181\,440$).

PLANTEAMIENTO Y FORMULACIÓN FORMAL

Un **problema de búsqueda** se define por la 5-tupla:

$$P = \langle S, , s_0, , A \; f \; C \rangle$$

donde Ses el conjunto de estados, $s_0$el estado inicial, Ael conjunto de acciones, $f : S \times A \to S$la función de transición y $C : S \times A \to \mathbb{R}^+$el costo. Un **espacio de búsqueda** es el grafo dirigido $G = (V, E)$con $V \subseteq S$y $E = \{(s, f(s,a)), \mid s \in V \; a \in A\}$, con peso $C(s,a)$.

Se modelan **dos problemas** con estructuras distintas (grafo físico ponderado vs. árbol combinatorio) para contrastar su complejidad.

PROBLEMA 1 — Navegación de un robot en almacén

Descripción: Un robot debe ir de la celda de carga (0,0) a la celda de despacho (3,3) en una rejilla 4×4 con obstáculos.

Componente	**Formalización**
Estado	$s = (f, c) \in \mathbb{Z}^2$, fila y columna del robot
Estado inicial	$s_0 = (0,0)$
Acciones	$A = \{N, S, E, O\}$
Transición	$f((f,c), N) = (f, -1c)$, etc., si la celda destino no es obstáculo
Costo	$C(s,a) = 1$para todo movimiento (uniforme)
Test de meta	$s = (3,3)$

Análisis de complejidad del espacio:

- $\mid S \mid = 16 - \mid obstáculos \mid$; con 2 obstáculos → $\mid S \mid = 14$
- Factor de ramificación máximo $b = 4$(interior); promedio ≈ 3
- $\mid E \mid \leq 14 \times 4 = 56$aristas dirigidas

Herramienta: Venice AI. **Uso:** andamiaje y revisión de la formulación. **Verificación:** se ejecutó el código y se contrastaron los conteos de estados contra los valores teóricos ($9!/2 = 181\,440$).

- Complejidad temporal de DFS/BFS: $O(b^d)$con $d_{max}' = 6$(distancia Manhattan) → ~$4^6 = 4096$ en el peor caso
- Complejidad espacial (BFS): $O(b^d)$= 4096 estados en frontera

PROBLEMA 2 — Puzle deslizante 8 (8-puzzle)

Descripción: Reordenar 8 fichas numeradas en una cuadrícula 3×3 deslizando una ficha adyacente al hueco, hasta alcanzar el orden objetivo.

Componente	**Formalización**
Estado	$s = (t_1, \ldots, t_9) \in \mathbb{N}^9$, permutación de $\{0..8\}$, con 0= hueco
Estado inicial	$s_0 = (1, 2, 3, 4, 0, 6, 7, 5, 8)$
Acciones	Deslizar la ficha vecina al hueco: $A = \{a, r, r, ibaabajoizqder\}$
Transición	Intercambio posicional entre hueco y ficha adyacente válida
Costo	$C(s, a) = 1$por deslizamiento
Test de meta	$s = (1, 2, 3, 4, 5, 6, 7, 8, 0)$

Análisis de complejidad del espacio:

- $\mid S \mid = 9! = 362\,880$permutaciones, pero solo la mitad son alcanzables (paridad) → $\mid S \mid = 181\,440$
- Factor de ramificación promedio $b \approx 2.67$(2 en esquinas, 3 en bordes, 4 en el centro)
- Profundidad máxima óptima del 8-puzzle: **31 movimientos**
- Complejidad temporal: $O(b^d) = 2.67^{31} \approx 3.8 \times 10^{13}$→ impracticable sin heurística (justifica A* con Manhattan/colisiones)
- Complejidad espacial: $O(b^d)$; el grafo completo requiere 181 440 nodos

Herramienta: Venice AI. **Uso:** andamiaje y revisión de la formulación. **Verificación:** se ejecutó el código y se contrastaron los conteos de estados contra los valores teóricos ($9!/2 = 181\,440$).

CONCLUSIONES

El modelado formal revela que ambos problemas, aunque triviales de enunciar, difieren en **órdenes de magnitud** en su espacio. La rejilla admite búsqueda exhaustiva; el puzle 8 no, lo que demuestra empíricamente por qué la búsqueda ciega colapsa y se vuelven necesarias heurísticas (siguiente actividad). NetworkX permitió además **validar la conectividad y la alcanzabilidad** del espacio antes de programar cualquier algoritmo, evitando búsquedas sobre estados inalcanzables.

REFERENCIAS (APA 7ª)

- Böhm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science.
- Russell, S., & Norvig, P. (2020). *Inteligencia artificial: Un enfoque moderno* (4ª ed.). Pearson.
- Wirsansky, E. (2020). *Hands-on genetic algorithms with Python*. Packt.
- NetworkX Developers. (2024). *NetworkX documentation*. https://networkx.org

Herramienta: Venice AI. **Uso:** andamiaje y revisión de la formulación. **Verificación:** se ejecutó el código y se contrastaron los conteos de estados contra los valores teóricos ($9!/2 = 181\,440$).